Tutorial completo de cursores en PL/pgSQL (PostgreSQL 18)

Este tutorial cubre todas las variantes de cursores en PL/pgSQL: explícitos e implícitos, con y sin variables `OUT`, con `RETURN TABLE`, `SETOF RECORD`, `RETURN QUERY`, `EXECUTE` dinámico, parámetros posicionales (`$1`, `$2`), y funciones JSON (`json_agg`, `json_build_object`, `to_json`, `to_jsonb`, `row_to_json`). Todos los ejemplos usan la base de datos `tienda` proporcionada.

Convención de nombres: todas las variables se declaran con el prefijo `v_` seguido del nombre de columna (`v_pedido_id`, `v_linea_id`, etc.), tal como solicitaste.

---

1. Cursores implícitos (FOR … IN … LOOP)

Un cursor implícito es el bucle `FOR` que recorre directamente el resultado de una consulta, sin declarar ni abrir un cursor explícito. Es la forma más sencilla y eficiente para la mayoría de casos.

Ejemplo 1.1: Cursor implícito con variable `RECORD`

Enunciado: Listar todos los pedidos con el nombre del cliente, mostrando `pedido_id`, `cliente` y `estado`.

sql

```
DO $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT p.pedido_id, c.nombre AS cliente, p.estado
        FROM pedidos p
        JOIN clientes c ON c.cliente_id = p.cliente_id
        ORDER BY p.pedido_id
    LOOP
        RAISE NOTICE 'Pedido: %, Cliente: %, Estado: %',
            v_registro.pedido_id, v_registro.cliente, v_registro.estado;
    END LOOP;
END $$;
```

Resultado:

text

```
NOTICE:  Pedido: 1, Cliente: Juan Pérez, Estado: pendiente
NOTICE:  Pedido: 2, Cliente: María López, Estado: pendiente
```

Ejemplo 1.2: Cursor implícito con %ROWTYPE

Enunciado: Recorrer la tabla `empleados` y mostrar nombre y salario usando una variable tipada con `%ROWTYPE`.

sql

```
DO $$
DECLARE
    v_empleado empleados%ROWTYPE;
BEGIN
    FOR v_empleado IN SELECT * FROM empleados ORDER BY id LOOP
        RAISE NOTICE 'Empleado: %, Salario: %', v_empleado.nombre,
v_empleado.salario;
    END LOOP;
END $$;
```

Resultado:

text

```
NOTICE:  Empleado: Ana Torres, Salario: 2800.00
NOTICE:  Empleado: Luis Martínez, Salario: 4500.00
... (8 filas)
```

Ejemplo 1.3: Cursor implícito con RETURN QUERY (sin cursor explícito)

Enunciado: Crear una función que devuelva todos los productos activos con su stock.

sql

```
CREATE OR REPLACE FUNCTION fn_productos_activos()
RETURNS TABLE (v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC, v_stock
INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT producto_id, nombre, precio, stock
        FROM productos
        WHERE activo = TRUE
        ORDER BY producto_id;
END $$;
```

Consulta:

sql

```
SELECT * FROM fn_productos_activos();
```

Resultado:

text

```
 v_producto_id | v_nombre           | v_precio | v_stock
---------------+--------------------+----------+---------
             1 | Teclado mecánico   |    79.90 |      50
             2 | Ratón inalámbrico  |    39.50 |     100
             3 | Monitor 27"        |   249.00 |      20
```

---

2. Cursores explícitos (DECLARE / OPEN / FETCH / CLOSE)

Un cursor explícito se declara con un nombre y una consulta, se abre, se recorren sus filas con `FETCH`, y se cierra. Es útil cuando necesitas control fino sobre la navegación (FETCH NEXT, PRIOR, ABSOLUTE, etc.) o cuando vas a devolver un `refcursor`.

Ejemplo 2.1: Cursor explícito básico con `FETCH` y variables simples

Enunciado: Recorrer los pedidos y mostrar el importe total de cada uno usando un cursor explícito.

sql

```
DO $$
DECLARE
    v_cursor CURSOR FOR
        SELECT p.pedido_id, SUM(lp.cantidad * lp.precio_unit) AS total
        FROM pedidos p
        JOIN lineas_pedido lp ON lp.pedido_id = p.pedido_id
        GROUP BY p.pedido_id
        ORDER BY p.pedido_id;
    v_pedido_id INTEGER;
    v_total NUMERIC;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_pedido_id, v_total;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Pedido %: total = %', v_pedido_id, v_total;
    END LOOP;
    CLOSE v_cursor;
END $$;
```

Resultado:

text

```
NOTICE:  Pedido 1: total = 158.90
NOTICE:  Pedido 2: total = 249.00
```

Ejemplo 2.2: Cursor explícito con variable `RECORD`

Enunciado: Usar un cursor explícito que devuelve filas completas de `clientes` y recorrerlas con una variable `RECORD`.

sql

```
DO $$
DECLARE
    v_cursor CURSOR FOR SELECT * FROM clientes ORDER BY cliente_id;
    v_registro RECORD;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_registro;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Cliente %: % (%)',
```

```
            v_registro.cliente_id, v_registro.nombre, v_registro.email;
    END LOOP;
    CLOSE v_cursor;
END $$;
```

Resultado:

text

```
NOTICE:  Cliente 1: Juan Pérez (juan@example.com)
NOTICE:  Cliente 2: María López (maria@example.com)
```

Ejemplo 2.3: Cursor explícito con `%ROWTYPE`

Enunciado: Recorrer la tabla `productos` con un cursor explícito y una variable `%ROWTYPE`.

sql

```
DO $$
DECLARE
    v_cursor CURSOR FOR SELECT * FROM productos ORDER BY producto_id;
    v_producto productos%ROWTYPE;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_producto;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Producto: % | Precio: % | Stock: %',
            v_producto.nombre, v_producto.precio, v_producto.stock;
    END LOOP;
    CLOSE v_cursor;
END $$;
```

Ejemplo 2.4: Cursor explícito con `RETURN TABLE` y variable `OUT`

Enunciado: Función que devuelve los pedidos con su total usando un cursor explícito y variables `OUT` declaradas en `RETURNS TABLE`.

sql

```
CREATE OR REPLACE FUNCTION fn_pedidos_totales()
RETURNS TABLE (v_pedido_id INTEGER, v_total NUMERIC)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor CURSOR FOR
        SELECT p.pedido_id, SUM(lp.cantidad * lp.precio_unit) AS total
        FROM pedidos p
        JOIN lineas_pedido lp ON lp.pedido_id = p.pedido_id
        GROUP BY p.pedido_id
        ORDER BY p.pedido_id;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_pedido_id, v_total;
        EXIT WHEN NOT FOUND;
        RETURN NEXT;
    END LOOP;
```

```
    CLOSE v_cursor;
END $$;
```
Consulta:

sql

```
SELECT * FROM fn_pedidos_totales();
```
Resultado:

text

```
 v_pedido_id | v_total
-------------+---------
           1 |  158.90
           2 |  249.00
```
Nota: cuando usas `RETURNS TABLE`, las columnas declaradas son variables `OUT` implícitas. El `RETURN NEXT` va **sin expresión**; las variables ya contienen los valores que se devolverán .

Ejemplo 2.5: Cursor explícito con `RETURNS SETOF RECORD`

Enunciado: Función que devuelve un conjunto de registros genéricos (`SETOF RECORD`) usando un cursor explícito. El llamador debe especificar la estructura con `AS`.

sql

```
CREATE OR REPLACE FUNCTION fn_pedidos_setof_record()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor CURSOR FOR
        SELECT pedido_id, cliente_id, estado, fecha_pedido
        FROM pedidos
        ORDER BY pedido_id;
    v_registro RECORD;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_registro;
        EXIT WHEN NOT FOUND;
        RETURN NEXT v_registro;
    END LOOP;
    CLOSE v_cursor;
END $$;
```
Consulta (el llamador define la estructura):

sql

```
SELECT * FROM fn_pedidos_setof_record() AS t(v_pedido_id INTEGER, v_cliente_id
INTEGER, v_estado TEXT, v_fecha TIMESTAMP);
```
Resultado:

text

```
 v_pedido_id | v_cliente_id | v_estado  |        v_fecha
-------------+--------------+-----------+------------------------
           1 |            1 | pendiente | 2026-10-20 12:00:00+00
```

```
          2 |           2 | pendiente | 2026-10-20 12:00:00+00
```

Ejemplo 2.6: `RETURN QUERY` con `RETURNS TABLE` (alternativa sin cursor explícito)

Enunciado: Misma funcionalidad que el ejemplo 2.4 pero usando `RETURN QUERY` directamente.

sql

```
CREATE OR REPLACE FUNCTION fn_pedidos_totales_query()
RETURNS TABLE (v_pedido_id INTEGER, v_total NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT p.pedido_id, SUM(lp.cantidad * lp.precio_unit)::NUMERIC
        FROM pedidos p
        JOIN lineas_pedido lp ON lp.pedido_id = p.pedido_id
        GROUP BY p.pedido_id
        ORDER BY p.pedido_id;
END $$;
```

Resultado: idéntico al ejemplo 2.4.

---

3. Cursores con parámetros y `EXECUTE` dinámico

Ejemplo 3.1: Cursor con parámetros (bound cursor)

Enunciado: Cursor que recibe un `cliente_id` como parámetro y devuelve sus pedidos.

sql

```
DO $$
DECLARE
    v_cursor CURSOR (v_cliente_id INTEGER) FOR
        SELECT pedido_id, estado, fecha_pedido
        FROM pedidos
        WHERE cliente_id = v_cliente_id
        ORDER BY pedido_id;
    v_pedido_id INTEGER;
    v_estado TEXT;
    v_fecha TIMESTAMP;
BEGIN
    OPEN v_cursor(1);
    LOOP
        FETCH v_cursor INTO v_pedido_id, v_estado, v_fecha;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Pedido % del cliente 1: % (%)', v_pedido_id, v_estado,
v_fecha;
    END LOOP;
    CLOSE v_cursor;
END $$;
```

Resultado:

text

```
NOTICE:  Pedido 1 del cliente 1: pendiente (2026-10-20 ...)
```
También puedes abrir con notación nombrada: `OPEN v_cursor(v_cliente_id := 1);`.

Ejemplo 3.2: `OPEN FOR EXECUTE` con parámetros posicionales (`$1`)

Enunciado: Cursor dinámico que filtra productos por precio mínimo usando `$1` y `USING`.

sql

```
DO $$
DECLARE
    v_cursor refcursor;
    v_registro RECORD;
    v_precio_min NUMERIC := 50.00;
BEGIN
    OPEN v_cursor FOR EXECUTE
        'SELECT producto_id, nombre, precio FROM productos WHERE precio >= $1
ORDER BY precio'
        USING v_precio_min;
    LOOP
        FETCH v_cursor INTO v_registro;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Producto: % | Precio: %', v_registro.nombre,
v_registro.precio;
    END LOOP;
    CLOSE v_cursor;
END $$;
```
Resultado:

text

```
NOTICE:  Producto: Teclado mecánico | Precio: 79.90
NOTICE:  Producto: Monitor 27" | Precio: 249.00
```
`$1` hace referencia al primer parámetro pasado con `USING`. El nombre de la tabla, si fuera dinámico, se insertaría con `format('%I', ...)`.

Ejemplo 3.3: `OPEN FOR EXECUTE` con `format()` y `$1` para nombre de tabla dinámico

Enunciado: Recorrer dinámicamente una tabla cuyo nombre se pasa como parámetro de función (usando `$1` para el valor de filtro y `format` para el identificador).

sql

```
CREATE OR REPLACE FUNCTION fn_contar_filas(tabla TEXT, filtro TEXT)
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor refcursor;
    v_contador BIGINT := 0;
    v_registro RECORD;
BEGIN
    OPEN v_cursor FOR EXECUTE
```

```
        format('SELECT * FROM %I WHERE estado = $1', tabla)
        USING filtro;
    LOOP
        FETCH v_cursor INTO v_registro;
        EXIT WHEN NOT FOUND;
        v_contador := v_contador + 1;
    END LOOP;
    CLOSE v_cursor;
    RETURN v_contador;
END $$;
```

Consulta:

sql

```
SELECT fn_contar_filas('pedidos', 'pendiente');
```

Resultado:

text

```
 fn_contar_filas
-----------------
               2
```

---

4. %ROWTYPE vs RECORD — diferencias prácticas

Característica	%ROWTYPE	RECORD
Estructura	Fija, definida en tiempo de compilación	Dinámica, se adapta al resultado
Seguridad	Verificación de tipos en compilación	Errores en tiempo de ejecución si se accede a campo inexistente
Uso típico	Filas de una tabla concreta	Resultados de joins o consultas variables

Ejemplo comparativo:

sql

```
DO $$
DECLARE
    v_cliente clientes%ROWTYPE;   -- estructura fija
    v_generico RECORD;            -- estructura dinámica
BEGIN
    -- %ROWTYPE: sabemos exactamente qué columnas tiene
    SELECT * INTO v_cliente FROM clientes WHERE cliente_id = 1;
    RAISE NOTICE 'Cliente: %', v_cliente.nombre;

    -- RECORD: puede recibir cualquier forma
    SELECT p.pedido_id, SUM(lp.cantidad) AS total_items
    INTO v_generico
    FROM pedidos p JOIN lineas_pedido lp ON lp.pedido_id = p.pedido_id
    WHERE p.pedido_id = 1
    GROUP BY p.pedido_id;
    RAISE NOTICE 'Pedido: %, Items: %', v_generico.pedido_id,
v_generico.total_items;
END $$;
```

---

5. Funciones JSON: `json_agg`, `json_build_object`, `to_json`, `to_jsonb`, `row_to_json`

5.1 `to_json(anyelement)` y `to_jsonb(anyelement)`

Enunciado: Convertir una fila de `clientes` a JSON usando `to_json` y `to_jsonb`.

sql

```
SELECT to_json(c) AS json_cliente FROM clientes c WHERE cliente_id = 1;
```

Resultado:

json

```
{"cliente_id":1,"nombre":"Juan
Pérez","email":"juan@example.com","fecha_alta":"2026-10-20"}
```

sql

```
SELECT to_jsonb(c) AS jsonb_cliente FROM clientes c WHERE cliente_id = 1;
```

Resultado (jsonb):

json

```
{"nombre": "Juan Pérez", "email": "juan@example.com", "cliente_id": 1,
"fecha_alta": "2026-10-20"}
```

`to_json` y `to_jsonb` convierten recursivamente arrays y tipos compuestos a JSON. Para escalares, producen un valor JSON escalar .

5.2 `row_to_json(record)`

Enunciado: Convertir una fila arbitraria (resultado de un join) a JSON.

sql

```
SELECT row_to_json(t) AS pedido_json
FROM (
    SELECT p.pedido_id, c.nombre AS cliente, p.estado,
           (SELECT SUM(cantidad * precio_unit) FROM lineas_pedido WHERE
pedido_id = p.pedido_id) AS total
    FROM pedidos p JOIN clientes c ON c.cliente_id = p.cliente_id
    WHERE p.pedido_id = 1
) t;
```

Resultado:

json

```
{"pedido_id":1,"cliente":"Juan Pérez","estado":"pendiente","total":158.90}
```

`row_to_json` es equivalente a `to_json` para tipos compuestos, con la opción de añadir saltos de línea (`pretty_bool`) .

5.3 `json_build_object(...)` + `json_agg(...)`

Enunciado: Construir un array JSON con todos los productos, donde cada producto sea un objeto con `id`, `nombre` y `precio`.

sql

```
SELECT json_agg(
    json_build_object(
        'id', producto_id,
        'nombre', nombre,
        'precio', precio
    )
) AS productos_json
FROM productos;
```

Resultado:

json

```
[
  {"id":1,"nombre":"Teclado mecánico","precio":79.90},
  {"id":2,"nombre":"Ratón inalámbrico","precio":39.50},
  {"id":3,"nombre":"Monitor 27\"","precio":249.00}
]
```

5.4 Combinación de cursor explícito + `json_agg` + `json_build_object`

Enunciado: Función que devuelve un JSON con los pedidos y sus líneas, usando un cursor explícito para recorrer pedidos y agregando las líneas con `json_agg`.

sql

```
CREATE OR REPLACE FUNCTION fn_pedidos_json()
RETURNS JSON
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor CURSOR FOR SELECT pedido_id, cliente_id, estado FROM pedidos ORDER
BY pedido_id;
    v_pedido_id INTEGER;
    v_cliente_id INTEGER;
    v_estado TEXT;
    v_resultado JSON;
    v_pedidos JSON[] := '{}';
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_pedido_id, v_cliente_id, v_estado;
        EXIT WHEN NOT FOUND;
        v_pedidos := array_append(v_pedidos,
            json_build_object(
                'pedido_id', v_pedido_id,
                'cliente_id', v_cliente_id,
                'estado', v_estado,
                'lineas', (
                    SELECT json_agg(json_build_object(
                        'producto_id', lp.producto_id,
                        'cantidad', lp.cantidad,
                        'precio_unit', lp.precio_unit,
```

```
                        'subtotal', lp.cantidad * lp.precio_unit
                    ))
                    FROM lineas_pedido lp WHERE lp.pedido_id = v_pedido_id
                )
            )
        );
    END LOOP;
    CLOSE v_cursor;
    RETURN array_to_json(v_pedidos);
END $$;
```

Consulta:

sql

```
SELECT jsonb_pretty(fn_pedidos_json()::jsonb);
```

Resultado (abreviado):

json

```
[
    {
        "estado": "pendiente",
        "lineas": [
            {"cantidad": 1, "subtotal": 79.90, "producto_id": 1, "precio_unit":
79.90},
            {"cantidad": 2, "subtotal": 79.00, "producto_id": 2, "precio_unit":
39.50}
        ],
        "pedido_id": 1,
        "cliente_id": 1
    },
    {
        "estado": "pendiente",
        "lineas": [
            {"cantidad": 1, "subtotal": 249.00, "producto_id": 3, "precio_unit":
249.00}
        ],
        "pedido_id": 2,
        "cliente_id": 2
    }
]
```

---

6. Tabla resumen de variantes de retorno

Mecanismo	Declaración	Cómo devolver	Cuándo usarlo
`RETURNS TABLE` + `RETURN NEXT`	`RETURNS TABLE (v_col1 tipo, ...)`	`RETURN NEXT;` (sin expresión)	Columnas fijas, legible
`RETURNS TABLE` + `RETURN QUERY`	`RETURNS TABLE (v_col1 tipo, ...)`	`RETURN QUERY SELECT ...;`	Consulta directa, más eficiente
`RETURNS SETOF RECORD`	`RETURNS SETOF RECORD`	`RETURN NEXT v_registro;`	Estructura desconocida hasta el llamador
`RETURNS SETOF`	`RETURNS SETOF`	`RETURN NEXT`	Devolver filas de una tabla

Mecanismo	Declaración	Cómo devolver	Cuándo usarlo
`tabla%ROWTYPE`	`productos`	`v_producto;`	concreta
`RETURNS JSON`	`RETURNS JSON`	`RETURN json_agg(...);`	Salida JSON agregada

7. Nota sobre `json_serialize`

`json_serialize` **no existe como función en PostgreSQL 18**. La serialización se logra con:

- `to_json(anyelement)` → `json`
- `to_jsonb(anyelement)` → `jsonb`
- `row_to_json(record)` → `json`
- `json_agg(...)` → `json`
- Cast explícito: `texto::json` o `jsonb::text`

Si necesitas convertir un `jsonb` a texto serializado, usa un cast: `mi_jsonb::text`.